

Crop and Weed Segmentation from UAV Images with a U-Net: Official Benchmark Results and Failure Analysis on PhenoBench

Muhammad Jawad
jawadkhangigyani@gmail.com

Abstract

I trained a U-Net with an ImageNet-pretrained ResNet34 encoder to label every pixel of drone images of sugar beet fields as soil, crop or weed, using the PhenoBench dataset. Weeds cover only 0.5% of the training pixels, so I trained with a combined Dice and cross-entropy loss. On the official hidden test set, scored by the benchmark server, the model reaches **84.72 mIoU** (soil 99.25, crop 93.33, weed 61.57). This is about 1.3 points below the ERFNet and DeepLabV3+ baselines published with the dataset. I then measured where the model fails. Detection depends strongly on plant size: only 31% of weeds smaller than 64 pixels (about 64 mm²) are detected, against 95% of weeds larger than 4,000 pixels. Plant–soil edges make up 6% of the pixels but contain 90% of the errors. Most importantly, weed IoU drops from 64.73 on test images of the training field to 36.43 on a different field recorded a year later, while crop IoU stays above 92. Code, notebooks and results are available at <https://github.com/JawadGigyani/CropSeg>.

1 Introduction

Spraying a whole field to remove weeds wastes herbicide, because weeds usually cover a small part of it. A robot or a precision sprayer can treat only the weeds, but first it has to know which pixels of an image are crop and which are weed. This is a semantic segmentation problem: every pixel gets a class label.

PhenoBench [1] is a public benchmark for this task. It provides high-resolution drone images of sugar beet fields with pixel-wise labels. Its test labels are hidden and can only be scored by an evaluation server, and its test set includes a field that does not appear in the training data. So it measures both accuracy and how well a model carries over to a new field.

In this project I trained a standard model, a U-Net [2] with a ResNet34 encoder [3], and evaluated it carefully. I did not propose a new method. My goal was to get an honest picture of how well a common baseline works on this benchmark, and where and why it fails. Concretely:

- I trained the model and submitted its predictions to the official test server, where it scores 84.72 mIoU (Section 5).
- I checked my evaluation code against PhenoBench’s official evaluation code, which gives identical numbers on the validation set (Section 4).
- I measured failures by plant size, error location, growth stage and field. Small plants and an unseen field turn out to be the main weaknesses (Section 6).

2 Dataset

I used PhenoBench v1.1.0 [1]. The images are 1024 × 1024 RGB tiles taken by a drone flying at about 21 m, which gives a ground sampling distance of about 1 mm per pixel. The training (1,407 images) and validation (772 images) sets come from one field recorded on three dates in 2020: May 15, May 26 and June 5. These dates are three growth stages. The test set (693 images) contains the same field and, in addition, a different

field recorded on four dates in 2021 (May 20, May 28, June 1 and June 10). The 2021 field never appears in training or validation.

The semantic labels have five values: soil, crop, weed, partial crop and partial weed. “Partial” marks plants with less than half of their area inside the image. I merged partial crop into crop and partial weed into weed, which is the same rule the official evaluation code applies. This gives three classes.

The classes are very imbalanced. Over all label masks, the training set is 87.7% soil, 11.9% crop and 0.50% weed, so soil outnumbers weed by about 176 to 1. The validation set is similar (89.9%, 9.6% and 0.53%). The imbalance is strongest at the earliest growth stage, where the soil-to-weed ratio in the training set is 517 to 1, compared with 66 to 1 at the latest stage.

3 Method

3.1 Model

I used a U-Net [2] with a ResNet34 encoder [3] pretrained on ImageNet [4], built with the Segmentation Models PyTorch library [5]. The encoder reduces a 512×512 input to a 16×16 feature map in five steps, and the decoder brings it back to full size, with skip connections between them. The model has 24.4 million parameters: 21.3 million in the encoder and 3.2 million in the decoder.

3.2 Input and augmentation

I resized each 1024×1024 image and its mask to 512×512 to fit the GPU memory and speed up training, and normalised colours with the ImageNet mean and standard deviation. During training I applied random horizontal and vertical flips, 90-degree rotations, brightness/contrast changes and colour jitter, using Albumentations [6]. Flips and rotations are safe here because the images are taken from directly above and have no natural “up”.

3.3 Loss

Cross-entropy treats every pixel equally, so with 88% soil pixels it is dominated by soil. Dice loss [7] is computed per class and then averaged, so the weed class counts as much as soil. I used the sum of an unweighted cross-entropy loss and a multi-class Dice loss.

3.4 Training

I trained for 40 epochs with batch size 8 (176 steps per epoch), using AdamW [8] with learning rate 3×10^{-4} and weight decay 10^{-4} , and a cosine learning-rate schedule [9] over the 40 epochs. Training ran in PyTorch [10] on a Google Colab T4 GPU. After every epoch I computed the validation IoU and kept the checkpoint with the best value, which came from epoch 39. Figure 1 shows the training curves. The validation loss has a few temporary spikes but no upward trend, and settles at about 0.144, so there is no sign of overfitting within 40 epochs.

3.5 Inference

The model always sees a 512×512 input. To score at the original resolution, I upsample its output scores to 1024×1024 with bilinear interpolation and take the class with the highest score at each pixel.

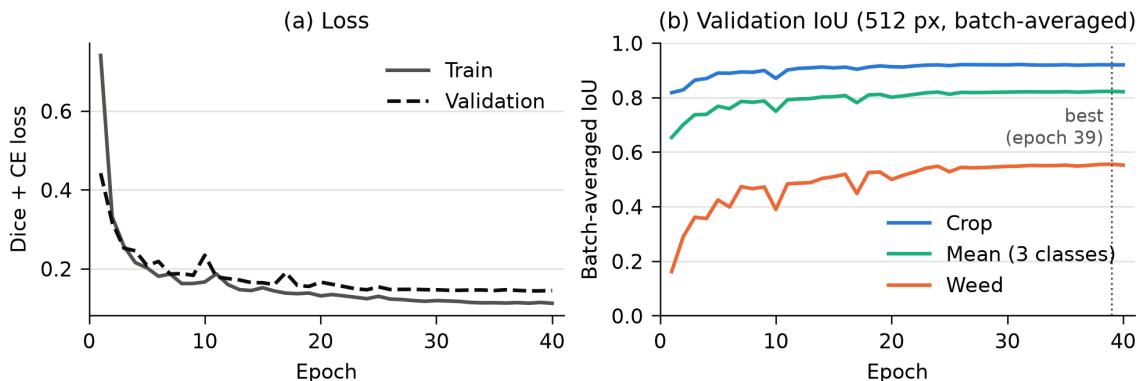


Figure 1. Training curves. (a) Training and validation loss. (b) Validation IoU as logged during training. These IoU values are averaged over batches (Section 4), so they are lower than the dataset-level numbers reported later.

4 Evaluation Protocol

4.1 Metric

For each class c , I count true positives, false positives and false negatives over *all* pixels of a split, and compute $\text{IoU}_c = \text{TP}_c / (\text{TP}_c + \text{FP}_c + \text{FN}_c)$. The mean IoU (mIoU) is the average over soil, crop and weed. This is the PhenoBench definition. To check my implementation, I scored my 1024×1024 validation predictions with PhenoBench’s official evaluation function. It returned exactly the same numbers as my code (soil 99.36, crop 94.87, weed 70.00, mIoU 88.08).

4.2 A lesson about averaging

During training I logged a different number: IoU computed for each batch of 8 images and then averaged over the 97 validation batches. For the selected checkpoint this gives 82.25, compared with 87.14 when all pixels are pooled at the same resolution. The gap comes from the weed class. A batch with only a few weed pixels gets a very low weed IoU from a handful of mistakes, and those batches pull the average down. An early version of my project reported this batch-averaged number. I now report only the dataset-level metric, although the checkpoint was still selected with the batch-averaged value.

4.3 Test set

I submitted predictions for all 693 test images to the PhenoBench evaluation server on Codabench¹. This used one of the ten submissions allowed per participant. The model was never trained, tuned or selected on test data.

4.4 Failure analysis

I ran three further measurements on the validation set at 1024×1024 . (1) *Plant size*: using PhenoBench’s plant instance masks, I counted a complete plant as detected if at least half of its pixels were predicted as its class. I left out partial plants, because their visible area is not their real size. (2) *Error location*: I assigned every pixel to one of four zones based on the ground truth: within 5 pixels of both a crop and a weed (crop–weed contact), within 5 pixels of a plant–soil boundary (edge), other plant pixels (interior), and other soil pixels. (3) *Growth stage*: I computed IoU separately for each recording date.

¹<https://www.codabench.org/competitions/14019>. The result is on the public leaderboard under the name “Jawad” (submission 965250). The server’s raw output, `scores.txt` and `detailed_results.html`, is in the repository as `Evaluation/scoring_result.zip`.

Table 1. Official PhenoBench test results (IoU in %). Baseline numbers are from Table III of [1]. My numbers are the server’s output for submission 965250 (Evaluation/scoring_result.zip).

Model	Test subset	mIoU	Soil	Crop	Weed
ERFNet [11]	all images	85.98	99.28	94.30	64.37
DeepLabV3+ [12]	all images	85.97	99.25	94.07	64.59
U-Net + ResNet34 (mine)	all images	84.72	99.25	93.33	61.57
	2020 field (seen in training)	85.84	99.33	93.46	64.73
	2021 field (unseen)	75.83	98.30	92.77	36.43

Table 2. Results by recording date (IoU in %). Validation at 1024×1024 ; test from the official server.

Field	Date	Validation		Test		
		mIoU	Weed	mIoU	Crop	Weed
2020	May 15	79.60	48.50	78.06	90.29	44.26
2020	May 26	83.55	57.80	84.29	93.60	60.56
2020	June 5	90.70	76.79	91.00	96.32	77.97
2021 (unseen)	May 20	–	–	68.15	69.86	35.04
2021 (unseen)	May 28	–	–	85.93	90.19	68.27
2021 (unseen)	June 1	–	–	71.97	87.43	29.90
2021 (unseen)	June 10	–	–	75.32	93.90	34.79

5 Results

Table 1 shows the official test results. Overall, the model reaches 84.72 mIoU. On the same test set, the two baselines reported in Table III of the PhenoBench paper [1], ERFNet [11] and DeepLabV3+ [12], reach 85.98 and 85.97. My model is about 1.3 points lower, and most of the difference is in the weed class (61.57 against about 64.5). On test images from the 2020 field, it reaches 85.84 mIoU. On the unseen 2021 field, it drops to 75.83 mIoU.

On the validation set, the model scores 88.08 mIoU at 1024×1024 (soil 99.36, crop 94.87, weed 70.00) and 87.14 at 512×512 . Scoring at the full resolution is slightly better, possibly because shrinking the label masks to 512×512 distorts thin leaf edges. The validation score is 3.4 points higher than the test score. Even on the 2020 test field alone, the model scores 2.2 points below validation. Part of this is expected, because I selected the checkpoint on the validation set, and the test images come from different crop rows. The unseen 2021 field then pulls the overall test score down by another 1.1 points.

Table 2 breaks the results down by recording date. In the 2020 field, the earliest date is the hardest and weed IoU improves strongly as the plants grow: on the test set it rises from 44.26 on May 15 to 77.97 on June 5, and the validation set shows the same pattern. In the 2021 field, the earliest date also has the lowest mIoU, and even crop IoU drops to 69.86 there. Weed IoU on the 2021 field does not follow a clear trend: it reaches 68.27 on May 28 but stays at about 30 to 35 on the other three dates.

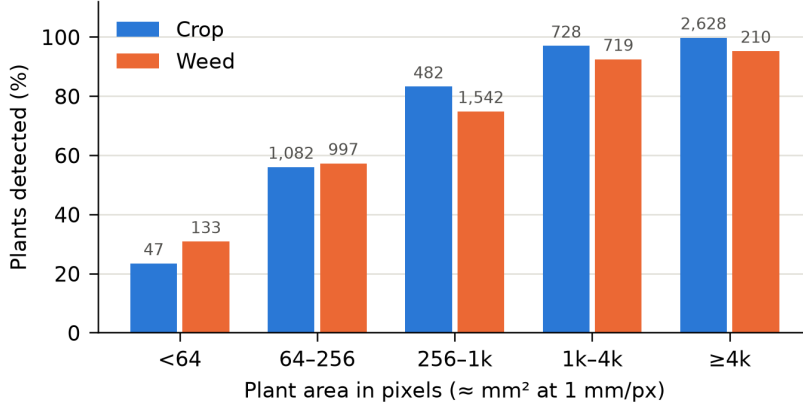


Figure 2. Share of complete plants detected (at least 50% of pixels predicted as the correct class), by plant area, on the validation set at 1024×1024 . Numbers above bars are plant counts.

Table 3. Where errors occur on the validation set (1024×1024 , zone width 5 pixels).

Zone	Share of pixels	Error rate	Share of errors	Share of crop–weed swaps
Crop–weed contact	0.06%	34.75%	3.15%	25.87%
Plant–soil edge	6.03%	9.40%	89.77%	47.08%
Plant interior	6.99%	0.43%	4.77%	27.04%
Open soil	86.92%	0.02%	2.30%	0.00%

6 Failure Analysis

6.1 Small plants are missed

Figure 2 shows the share of plants detected by size. Only 30.8% of the 133 weeds smaller than 64 pixels are detected, against 95.2% of weeds larger than 4,000 pixels. Crops follow the same pattern (23.4% for the smallest, 99.8% for the largest). When a small plant is missed, most of its pixels are predicted as soil. Small plants are more common among weeds than crops: 31% of the 3,601 complete weeds are under 256 pixels, against 23% of the 4,967 complete crops. So this weakness hurts the weed score more. One likely cause is the downsampling to 512×512 , which leaves a weed of 8 by 8 mm with only about 4 by 4 pixels. I have not tested this yet.

6.2 Errors sit on plant outlines

Table 3 shows where the errors are. Plant–soil edges are 6.03% of the pixels but hold 89.77% of all errors. Pixels where crops and weeds touch have a very high error rate (34.75%), but these zones are so rare (0.06% of pixels) that they hold only 3.15% of the errors. They do account for a quarter of all crop–weed swaps. So crop–weed boundaries are a real but small problem; outlines in general are the big one. The confusion matrix points the same way: 71% of the weed pixels the model misses are predicted as soil, not as crop, and 71% of the pixels it wrongly calls weed are soil.

6.3 The unseen field is the biggest weakness

On the 2021 field, crop IoU barely changes (92.77 against 93.46 on the 2020 field), but weed IoU almost halves (36.43 against 64.73). The model clearly recognises sugar beets in a new field but not that field’s weeds. The PhenoBench authors describe the 2021 field as having potentially different weeds [1], and my

model only ever saw weeds from one field. Weeds also vary much more in shape and colour than a single crop species. I think this is the most likely reason, but I could not test it, because the test labels are hidden and there are no per-species labels.

7 Limitations

I trained a single model with no fixed random seed, so I cannot give error bars. I did not run an ablation, so I cannot show that Dice plus cross-entropy is better than cross-entropy alone; I chose it for the class imbalance. I selected the checkpoint with the batch-averaged validation IoU, not the dataset-level metric.

8 Conclusion

A standard U-Net with a pretrained ResNet34 encoder reaches 84.72 mIoU on the official PhenoBench test set, slightly below the published baselines. Weeds are the hard class, for two measured reasons: small weeds are often missed, and weed accuracy almost halves on a field the model has not seen. Most errors sit on plant outlines rather than where crops and weeds touch.

Reproducibility

Everything reported here is in the repository (<https://github.com/JawadGigyani/CropSeg>): the training run and its log in `training.ipynb`; the validation results, official cross-check, failure analysis and test submission in `evaluation.ipynb` (numbers in `Evaluation/eval_results.json`); and the class distribution, checkpoint details and parameter count in `dataset_and_model_stats.ipynb`. The repository also links the trained weights.

References

- [1] J. Weyler, F. Magistri, E. Marks, et al. PhenoBench: A large dataset and benchmarks for semantic image interpretation in the agricultural domain. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [2] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *MICCAI*, 2015.
- [3] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
- [4] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009.
- [5] P. Iakubovskii. Segmentation Models PyTorch. GitHub repository, https://github.com/qubvel/segmentation_models.pytorch, 2019.
- [6] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, et al. Albumentations: Fast and flexible image augmentations. *Information*, 11(2):125, 2020.
- [7] F. Milletari, N. Navab, and S.-A. Ahmadi. V-Net: Fully convolutional neural networks for volumetric medical image segmentation. In *3DV*, 2016.
- [8] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.
- [9] I. Loshchilov and F. Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017.
- [10] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.
- [11] E. Romera, J. M. Álvarez, L. M. Bergasa, et al. ERFNet: Efficient residual factorized ConvNet for real-time semantic segmentation. *IEEE Transactions on Intelligent Transportation Systems*, 19(1):263–272, 2018.
- [12] L.-C. Chen, Y. Zhu, G. Papandreou, et al. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *ECCV*, 2018.